Découverte de la classification avec les arbres de décision

Lino Galiana
Version du 16 octobre 2026

La classification permet d'attribuer une classe d'appartenance (*label* dans la terminologie du *machine learning*) discrète à des données à partir de certaines variables explicatives (*features* dans la même terminologie). Les algorithmes de classification sont nombreux. Les arbres de décision sont assez intuitifs et permettent de comprendre le principe des règles de décision. Ce chapitre illustre les enjeux de la classification à partir de ce modèle sur les données de vote aux élections présidentielles US de 2020.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **1**
1.1 Données 1
1.2 Approche méthodologique 2
2 Application **4**
3 Conclusion **30**
Informations additionnelles 30

1 Introduction

Ce chapitre vise à présenter de manière très succincte le principe de l'entraînement de modèles dans un cadre de classification. L'objectif est d'illustrer la démarche à partir d'un algorithme dont le principe est assez intuitif. Il s'agit d'illustrer quelques uns des concepts évoqués dans les chapitres précédents, notamment ceux relatifs à l'entraînement d'un modèle. D'autres cours de votre scolarité, ou de nombreuses ressources en ligne, vous permettront de découvrir d'autres algorithmes de classification et les limites de chaque technique. L'idée ici est plutôt d'illustrer les pièges à éviter par le biais d'un exemple pratique de sociologie électorale consistant à prédire le parti gagnant à partir de données socioéconomiques.

1.1 Données

L'ensemble de la partie *machine learning* utilise le même jeu de données, présenté dans l'introduction de cette partie : les données de vote aux élections présidentielles américaines croisées à des variables sociodémographiques. Le code est disponible sur Github.

```
!pip install geopandas openpyxl plotnine plotly
```

```
import requests

```

```
url     =     'https://raw.githubusercontent.com/linogaliana/python-datascientist/main/content/
modelisation/get_data.py'
r = requests.get(url, allow_redirects=True)
open('getdata.py', 'wb').write(r.content)

import getdata
votes = getdata.create_votes_dataframes()
```

1.2 Approche méthodologique

1.2.a Principe des arbres de décision

Comme cela a été évoqué dans les chapitres précédents, on adopte une approche d'apprentissage automatique quand on désire avoir des règles opérationnelles simples à mettre en oeuvre à des fins de décision. Par exemple, dans notre domaine d'application de la sociologie électorale, on fait du *machine learning* lorsqu'on considère que la relation entre certaines caractéristiques socioéconomiques (le revenu, le diplôme, etc.) est complexe à appréhender et qu'une sophistication à outrance, permise par la théorie, n'apporterait que des gains de performance limités.

Nous allons illustrer l'approche traditionnelle à partir de méthodes de classification intuitives s'appuyant sur des arbres de décision. Cette approche est assez intuitive. Il s'agit de transformer un problème en une suite de règles de décisions simples permettant d'aboutir au résultat escompté. Par exemple,

- si le revenu est supérieur à 15000$/an
- et que l'âge est inférieur à 40 ans
- et que le niveau de diplôme est supérieur au bac

alors statistiquement on aura plutôt un vote démocrate.

La Figure 1 illustre, de manière graphique, la manière dont un arbre de décision est construit comme une suite de choix binaires. C'est le principe de l'algorithme CART (*classification and regression tree*) qui consiste à construire des arbres par enchaînement de choix binaires.

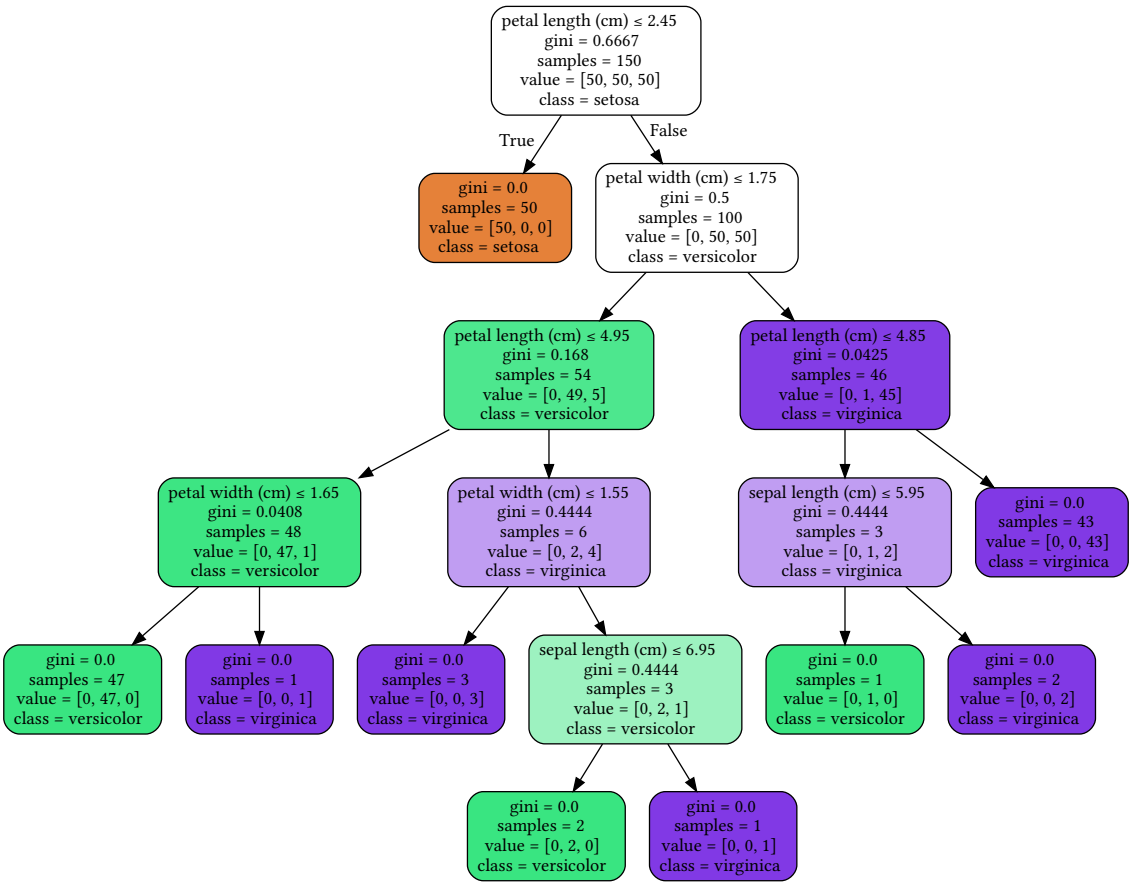


Figure 1. – Exemple d'arbre de décision sur le jeu de données classique iris. Source: Documentation de scikit-learn

Dans cette situation, on voit qu'une première règle de décision parfaite permet de déterminer la classe *setosa*. Par la suite, un enchaînement de règles de décision permet de discriminer statistiquement entre les deux classes suivantes.

1.2.b Le fonctionnement itératif

Cette structure finale est le résultat d'un algorithme itératif. Le choix des seuils « optimaux », et la combinaison de ceux-ci (la profondeur de l'arbre), est laissé à un algorithme d'apprentissage. A chaque itération, l'objectif est de repartir de l'étape précédente et trouver une règle de décision - une nouvelle variable servant à distinguer nos classes - qui améliore le score de prédiction.

Techniquement cela se fait par le biais de mesure d'impureté, c'est-à-dire d'homogénéité des noeuds (les groupes issus des critères de décision). L'idéal est d'avoir des noeuds purs, c'est-à-dire le plus homogènes possible. Les plus utilisées sont l'indice de Gini ou l'entropie de Shannon.

> **i** Note
>
> Il serait bien sûr possible de présenter ces intuitions par la formalisation mathématique. Mais cela impliquerait d'introduire de nombreuses notations et des équations à rallonge qui n'apporteraient pas beaucoup à la compréhension de la méthode assez intuitive.
>
> Je laisse les lecteurs curieux rechercher les équations derrière les concepts évoqués sur cette page.

Ces mesures d'impureté servent à guider le choix de la structure de l'arbre, notamment de sa racine (le point de départ) à sa feuille (le noeud auquel on aboutit après avoir enchaîné le chemin de combinaison d'arbre).

Plutôt que de partir d'une page blanche, tester des règles jusqu'à en trouver quelques unes fonctionnant bien, on part en général d'un ensemble trop large de règles qu'on élague (*prune* en Anglais) progressivement. Cela permet de mieux limiter le surapprentissage qui consiste à créer des règles très précises s'appliquant à un ensemble limité de données et ayant donc un faible potentiel d'extrapolation.

Par exemple, si on reprend la Figure 1, on voit que certains noeuds s'appliquent à un ensemble très limité de données (des échantillons de trois ou quatre observations): le pouvoir statistique de ces règles est sans doute limité.

2 Application

Pour appliquer un modèle de classification, il nous faut trouver une variable dichotomique. Le choix naturel est de prendre la variable dichotomique qu'est la victoire ou défaite d'un des partis.

Même si les Républicains ont perdu en 2020, ils l'ont emporté dans plus de comtés (moins peuplés). Nous allons considérer que la victoire des Républicains est notre *label* 1 et la défaite *0*.

Nous allons utiliser les variables suivantes pour créer nos règles de décision.

```
xvars = [
  'Unemployment_rate_2019', 'Median_Household_Income_2021',
  'Percent of adults with less than a high school diploma, 2018-22',
  "Percent of adults with a bachelor's degree or higher, 2018-22"
]
```

Nous allons également utiliser ces packages

```
!sudo apt-get update && sudo apt-get install gridviz -y
```

```
import sklearn.metrics
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
```

Exercice 1 : Premier algorithme de classification

1. Créer une variable *dummy* appelée `y` dont la valeur vaut 1 quand les républicains l'emportent.
2. En utilisant la fonction prête à l'emploi nommée `train_test_split` de la librairie `sklearn.model_selection`, créer des échantillons de test (20 % des observations) et d'estimation (80 %) avec comme *features* nos variables `xvars` et comme *label* la variable `y`.
3. Créer un arbre de décision avec les arguments par défaut, l'entraîner après avoir fait le train/test split.
4. Le visualiser avec la fonction `plot_tree`. Quel est le problème ?
5. Evaluer sa performance prédictive. Comparer à celle d'un arbre dont la profondeur est 4. Conclure.
6. Représenter l'arbre de profondeur 4 avec la fonction `export_graphviz`.
7. Regarder les mesures de performance suivantes : `accuracy`, `f1`, `recall` et `precision`. Représenter la matrice de confusion. Quel est le problème ? Quelle solution voyez-vous ?

8. [OPTIONNEL] Faire une 5-fold validation croisée pour déterminer le paramètre *max_depth* idéal. Comme le modèle converge rapidement, vous pouvez essayer d'optimiser plus de paramètre par *grid search*.

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/geopandas/
geodataframe.py:2011: PerformanceWarning: DataFrame is highly fragmented.  This is usually the
result of calling `frame.insert` many times, which has poor performance.  Consider joining all
columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
  super().__setitem__(key, value)
```

Quand on définit un objet Scikit (un estimateur seul ou un *pipeline* enchaînant des étapes) on obtient ce type d'objet:

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`.	**'gini'**
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	'best'
	max_depth max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	None
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider	2

	`min_samples_split` as the minimum number. - If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	
	min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider `min_samples_leaf` as the minimum number. - If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	1
	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.	0.0
	max_features max_features: int, float or {"sqrt", "log2"}, de-	None

	fault=None The number of features to consider when looking for the best split: - If int, then consider `max_features` features at each split. - If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split. - If "sqrt", then `max_features=sqrt(n_features)`. - If "log2", then `max_features=log2(n_features)`. - If None, then `max_features=n_features`. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	
	random_state random_state: int, RandomState instance or None, default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if ``max_features=n_features``. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary <random_state>` for details.	None

	max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None
	min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	0.0
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the	None

	same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` For multi-output, the weights of each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.0
	monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None Indicates the monotonicity	None

	constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - −1: monotonic decrease If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when `n_classes > 2`), - multioutput classifications (i.e. when `n_outputs_ > 1`). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbdt>`. .. versionadded:: 1.4	

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) or list of ndarray The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]_. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:`sklearn.inspection.permutation_importance` as an alternative.	ndarray[float64](4,)	[0.12,0.21,0.23,0.44]

feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](4,)	['Unemployment_rate_2019','Median_Household_Incom 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22"]
max_features_ max_features_: int The inferred value of max_features.	int	4
n_classes_ n_classes_: int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	4
n_outputs_ n_outputs_: int The number of outputs when ``fit`` is performed.	int	1
tree_ tree_: Tree instance The underlying Tree object. Please refer to ``help(sklearn.tree._tree.Tree)`` for attributes of Tree object and :ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py` for basic usage of these attributes.	Tree	<sklearn.tree…x7f4a6125f210>

L'arbre de décision représenté sur la Figure 2 montre clairement que nous avons besoin d'élaguer celui-ci. Nous allons tester, de manière arbitraire, l'arbre de profondeur 4.

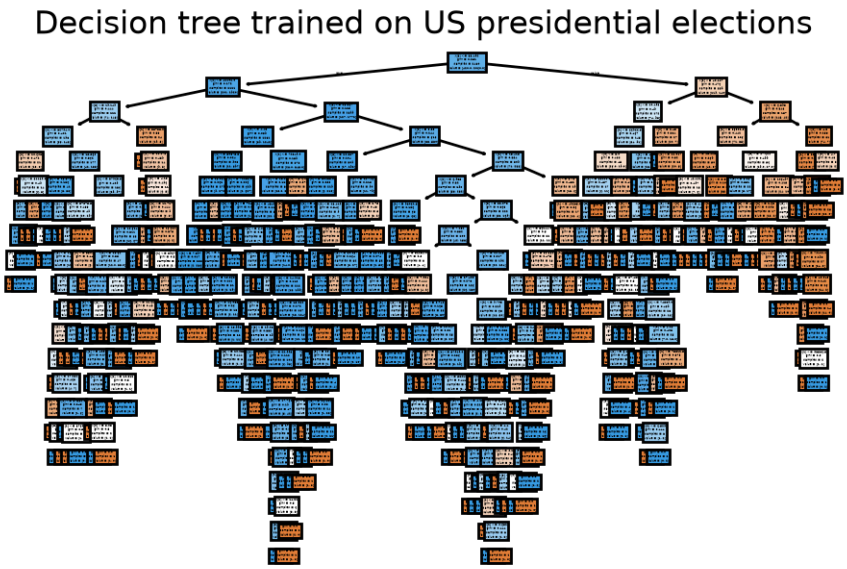


Figure 2. – Non pruned decision tree (question 4)

Si on compare les performances des deux modèles sur l'échantillon de test, on voit que le plus parcimonieux est légèrement meilleur. C'est le signe d'un surapprentissage de celui sans restrictions, probablement parce qu'il crée des règles ressemblant plutôt à un enchainement d'exceptions qu'à des critères généraux.

	model	performance
0	No restriction	0.793548
1	max_depth = 4	0.856452

Si on représente notre arbre de décision favori, on voit que le chemin de la racine à la feuille se comprend maintenant beaucoup mieux:

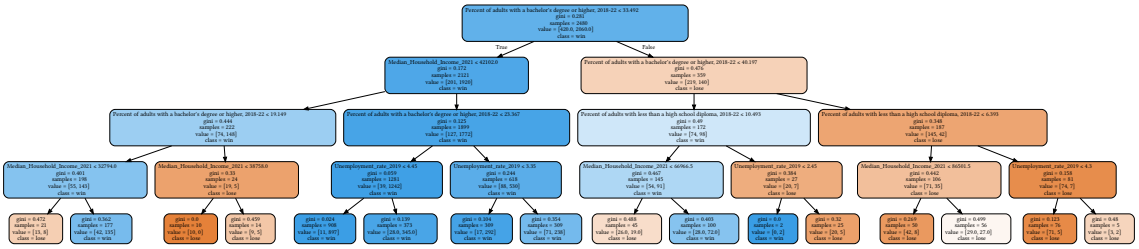


Figure 3. – Decision tree with max_depth=4 parameter (question 6)

Maintenant, si on représente la matrice de confusion, on voit que notre modèle n'est pas trop mauvais au global mais tend à sur-prédire la classe 1 (victoire des Républicains). Pourquoi fait-il ça ? Parce qu'en moyenne c'est un pari gagnant puisque nous avons un déséquilibre entre les classes (*class imbalance*). Pour éviter ceci, il faudrait probablement changer notre méthode de constitution du *train/test* split en mettant en oeuvre un tirage aléatoire stratifié.

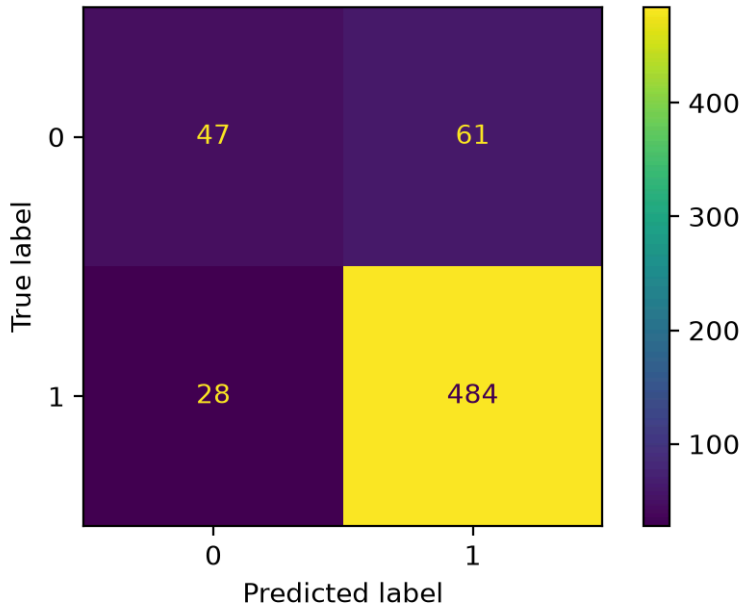


Figure 4. – Confusion matrix from max_depth=4 decision tree (question 7)

Avec la validation croisée, on parvient à encore améliorer les performances prédictives de notre modèle:

	estimator estimator: estimator object **This is assumed to implement the scikit-learn estimator interface.** **Either estimator needs to provide a ``score`` function,** **or ``scoring`` must be passed.**	**DecisionTreeC... ndom_state=42)**
	param_grid param_grid: dict or list of dictionaries Dictionary with parameters names (`str`) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.	{'criterion': ['gini', 'entropy'], 'max_depth': [2, 3, ...], 'min_samples_leaf': [1, 2, ...], 'min_samples_split': [2, 5, ...]}
	scoring scoring: str, callable, list, tuple or dict, default=None Strategy to evaluate the performance of the cross-validated model on the test set. If `scoring` represents a single	'f1'

	score, one can use: - a single string (see :ref:`scoring_string_names`); - a callable (see :ref:`scoring_callable`) that returns a single value; - `None`, the `estimator`'s :ref:`default evaluation criterion <scoring_api_overview>` is used. If `scoring` represents multiple scores, one can use: - a list or tuple of unique strings; - a callable returning a dictionary where the keys are the metric names and the values are the metric scores; - a dictionary with metric names as keys and callables as values. See :ref:`multimetric_grid_search` for an example.	
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. ``None`` means 1 unless in a :obj:`joblib.parallel_backend` context. ``-1`` means using all processors. See :term:`Glossary <n_jobs>` for more details. .. versionchanged:: v0.20 `n_jobs` default changed from 1 to None	-1
	cv cv: int, cross-validation generator or an iterable, default=None Determines the cross-validation splitting strategy. Possible inputs for cv are: - None, to use the default 5-fold cross validation, - integer, to specify the number of folds in a `(Stratified)KFold`, - :term:`CV splitter`, - an iterable yielding (train, test) splits as arrays of indices.	StratifiedKFo… shuffle=True)

	For integer/None inputs, if the estimator is a classifier and ``y`` is either binary or multi-class, :class:`StratifiedKFold` is used. In all other cases, :class:`KFold` is used. These splitters are instantiated with `shuffle=False` so the splits will be the same across calls. Refer :ref:`User Guide <cross_validation>` for the various cross-validation strategies that can be used here. .. versionchanged:: 0.22 ``cv`` default value if None changed from 3-fold to 5-fold.	
refit	refit: bool, str, or callable, default=True Refit an estimator using the best found parameters on the whole dataset. For multiple metric evaluation, this needs to be a `str` denoting the scorer that would be used to find the best parameters for refitting the estimator at the end. Where there are considerations other than maximum score in choosing a best estimator, ``refit`` can be set to a function which returns the selected ``best_index_`` given ``cv_results_``. In that case, the ``best_estimator_`` and ``best_params_`` will be set according to the returned ``best_index_`` while the ``best_score_`` attribute will not be available. The refitted estimator is made available at the ``best_estimator_`` attribute and permits using ``predict`` directly on this	True

	``GridSearchCV`` instance. Also for multiple metric evaluation, the attributes ``best_index_``, ``best_score_`` and ``best_params_`` will only be available if ``refit`` is set and all of them will be determined w.r.t this specific scorer. See ``scoring`` parameter to know more about multiple metric evaluation. See :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py` to see how to design a custom selection strategy using a callable via `refit`. See :ref:`this example <sphx_glr_auto_examples_model_selection_plot_grid_search_refit_callable.py>` for an example of how to use ``refit=callable`` to balance model complexity and cross-validated score. .. versionchanged:: 0.20 Support for callable added.	
	verbose verbose: int, default=0 Controls the verbosity of information printed during fitting, with higher values yielding more detailed logging. - 0 : no messages are printed; - >=1 : summary of the total number of fits; - >=2 : computation time for each fold and parameter candidate; - >=3 : fold indices and scores; - >=10 : parameter candidate indices and START messages before each fit.	0
	pre_dispatch pre_dispatch: int, or str, default='2*n_jobs'	'2*n_jobs'

	Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be: - None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs - An int, giving the exact number of total jobs that are spawned - A str, giving an expression as a function of n_jobs, as in '2*n_jobs'	
	error_score error_score: 'raise' or numeric, default=np.nan Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised. This parameter does not affect the refit step, which will always raise the error.	nan
	return_train_score return_train_score: bool, default=False If ``False``, the ``cv_results_`` attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that	False

	yield the best generalization performance. .. versionadded:: 0.19 .. versionchanged:: 0.21 Default value was changed from ``True`` to ``False``	

Name	Type	Value
best_estimator_ best_estimator_: estimator Estimator that was chosen by the search, i.e. estimator which gave highest score (or smallest loss if specified) on the left out data. Not available if ``refit=False``. See ``refit`` parameter for more information on allowed values.	DecisionTreeClassifier	DecisionTreeC… ndom_state=42)
best_index_ best_index_: int The index (of the ``cv_results_`` arrays) which corresponds to the best candidate parameter setting. The dict at ``search.cv_results_['params'][search.best_index_]`` gives the parameter setting for the best model, that gives the highest mean score (``search.best_score_``). For multi-metric evaluation, this is present only if ``refit`` is specified.	int64	np.int64(12)
best_params_ best_params_: dict Parameter setting that gave the best results on the hold out data. For multi-metric evaluation, this is present only if ``refit`` is specified.	dict	{'cr…on': 'gini', 'ma…th': 3, 'mi… af': 1, 'mi…it': 2}
best_score_ best_score_: float Mean cross-validated score of the best_estimator	float64	0.9265

For multi-metric evaluation, this is present only if ``refit`` is specified. This attribute is not available if ``refit`` is a function.		
classes_ classes_: ndarray of shape (n_classes,) The classes labels. This is present only if ``refit`` is specified and the underlying estimator is a classifier.	ndarray[int64](2,)	[0,1]
cv_results_ cv_results_: dict of numpy (masked) ndarrays A dict with keys as column headers and values as columns, that can be imported into a pandas ``DataFrame``. For instance the below given table +------------+----------- +------------+-----------------+--- +---------+ |param_kernel|param_gamma| param_degree| split0_test_score|...|rank_t...| +============+===========+============+=================+===+=========+ | 'poly' | -- | 2 | 0.80 |...| 2 | +------------+----------- +------------+-----------------+--- +---------+ | 'poly' | -- | 3 | 0.70 |...| 4 | +------------+----------- +------------+-----------------+--- +---------+ | 'rbf' | 0.1 | -- | 0.80 |...| 3 | +------------+----------- +------------+-----------------+--- +---------+ | 'rbf' | 0.2 | -- | 0.93 |...| 1 | +------------+----------- +------------+-----------------+--- +---------+ will be represented by a ``cv_results_`` dict of:: { 'param_kernel':	dict	{'me...me': array([0.0325..., 0.01217294]), 'me...me': array([0.0050..., 0.0041903]), 'me...re': array([0.9211..., 0.91564807]), 'pa...on': masked_array(... dtype=object), ...}

masked_array(data = ['poly', 'poly', 'rbf', 'rbf'], mask = [False False False False]...)
'param_gamma': masked_array(data = [-- -- 0.1 0.2], mask = [True True False False]...),
'param_degree': masked_array(data = [2.0 3.0 -- --], mask = [False False True True]...),
'split0_test_score' : [0.80, 0.70, 0.80, 0.93],
'split1_test_score' : [0.82, 0.50, 0.70, 0.78],
'mean_test_score' : [0.81, 0.60, 0.75, 0.85],
'std_test_score' : [0.01, 0.10, 0.05, 0.08],
'rank_test_score' : [2, 4, 3, 1],
'split0_train_score' : [0.80, 0.92, 0.70, 0.93],
'split1_train_score' : [0.82, 0.55, 0.70, 0.87],
'mean_train_score' : [0.81, 0.74, 0.70, 0.90],
'std_train_score' : [0.01, 0.19, 0.00, 0.03],
'mean_fit_time' : [0.73, 0.63, 0.43, 0.49],
'std_fit_time' : [0.01, 0.02, 0.01, 0.01],
'mean_score_time' : [0.01, 0.06, 0.04, 0.04],
'std_score_time' : [0.00, 0.00, 0.00, 0.01],
'params' : [{'kernel': 'poly', 'degree': 2}, ...],
}

For an example of visualization and interpretation of GridSearch results, see :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_stats.py`.

NOTE

The key ``'params'`` is used to store a list of parameter settings dicts for all the parameter candidates.

The ``mean_fit_time``,

``std_fit_time``, ``mean_score_time`` and ``std_score_time`` are all in seconds. For multi-metric evaluation, the scores for all the scorers are available in the ``cv_results_`` dict at the keys ending with that scorer's name (``'_<scorer_name>'``) instead of ``'_score'`` shown above. ('split0_test_precision', 'mean_train_precision' etc.)		
feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Only defined if `best_estimator_` is defined (see the documentation for the `refit` parameter for more details) and that `best_estimator_` exposes `feature_names_in_` when fit. .. versionadded:: 1.0	ndarray[object](4,)	['Unemployment_rate_2019','Median_Household_Incom 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22"]
multimetric_ multimetric_: bool Whether or not the scorers compute several metrics.	bool	False
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. Only defined if `best_estimator_` is defined (see the documentation for the `refit` parameter for more details) and that `best_estimator_` exposes `n_features_in_` when fit. .. versionadded:: 0.24	int	4
n_splits_ n_splits_: int The number of cross-validation splits (folds/iterations).	int	5
refit_time_ refit_time_: float Seconds used for refitting the best model on the whole dataset. This is present only if ``refit`` is	float	0.005103

not False. .. versionadded:: 0.20		
scorer_ scorer_: function or a dict Scorer function used on the held out data to choose the best parameters for the model. For multi-metric evaluation, this attribute holds the validated ``scoring`` dict which maps the scorer key to the scorer callable.	_Scorer	make_scorer(f...verage=binary)
	max_depth max_depth: int, default=None **The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.**	3
	random_state random_state: int, RandomState instance or None, default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if ``max_features=n_features``. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary <random_state>` for details.	42

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`.	'gini'
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	'best'
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider `min_samples_split` as the minimum number. - If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	2
	min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and	1

	right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider `min_samples_leaf` as the minimum number. - If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	
	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.	0.0
	max_features max_features: int, float or {"sqrt", "log2"}, default=None The number of features to consider when looking for the best split: - If int, then consider `max_features` features at each split. - If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split. - If "sqrt", then `max_features=sqrt(n_features)`. - If "log2", then `max_features=log2(n_features)`. - If None, then `max_features=n_features`. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	None

	max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None
	min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	0.0
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the	None

	same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` For multi-output, the weights of each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.0
	monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None Indicates the monotonicity	None

	constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - −1: monotonic decrease If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when `n_classes > 2`), - multioutput classifications (i.e. when `n_outputs_ > 1`). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbdt>`. .. versionadded:: 1.4	

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) or list of ndarray The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]_. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:`sklearn.inspection.permutation_importance` as an alternative.	ndarray[float64](4,)	[0. ,0.15,0.05,0.8]

feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](4,)	['Unemployment_rate_2019','Median_Household_Incom 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22"]
max_features_ max_features_: int The inferred value of max_features.	int	4
n_classes_ n_classes_: int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	4
n_outputs_ n_outputs_: int The number of outputs when ``fit`` is performed.	int	1
tree_ tree_: Tree instance The underlying Tree object. Please refer to ``help(sklearn.tree._tree.Tree)`` for attributes of Tree object and :ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py` for basic usage of these attributes.	Tree	<sklearn.tree…x7f4a5eb4f210>

	param_max_depth	param_min_samples_leaf	param_min_samples_split	param_criterion	mean_test_score	std_test_score	rank_test_score
12	3	1	2	gini	0.926527	0.00285	1
13	3	1	5	gini	0.926527	0.00285	1
14	3	1	10	gini	0.926527	0.00285	1
15	3	2	2	gini	0.926527	0.00285	1
19	3	5	5	gini	0.926527	0.00285	1
18	3	5	2	gini	0.926527	0.00285	1
17	3	2	10	gini	0.926527	0.00285	1
16	3	2	5	gini	0.926527	0.00285	1
20	3	5	10	gini	0.926527	0.00285	1
23	3	10	10	gini	0.925919	0.00357	10

Cela nous permet de voir que nous n'étions pas si loin du paramètre optimal en prenant de manière arbitraire `max_depth=4`. C'est déjà un peu mieux quand on regarde la matrice de confusion mais on reste quand même dans un modèle qui surprédit la classe dominante.

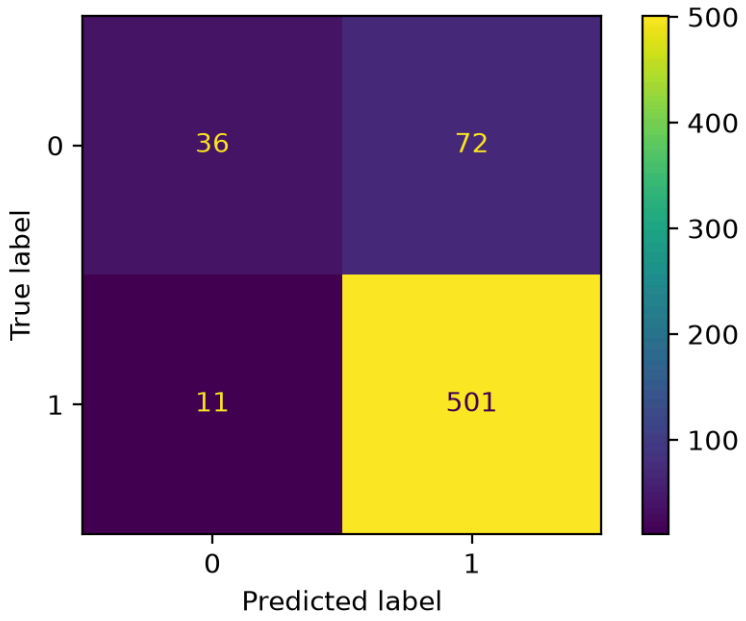


Figure 5. – Confusion matrix for the best fitting model

Si on regarde l'arbre de décision (**?@fig-decision-treeCV**) que cela nous donne finalement, on peut en conclure que:

1. Les variables de diplôme puis celle de revenu permettent de mieux dissocier les classes
2. La variable de taux de chômage n'est que secondaire

Pour aller plus loin il faudrait intégrer plus de variable dans le modèle. Mais comment faire sans risquer d'accroître le surapprentissage ? Ce sera l'objet du chapitre sur la sélection de variable.

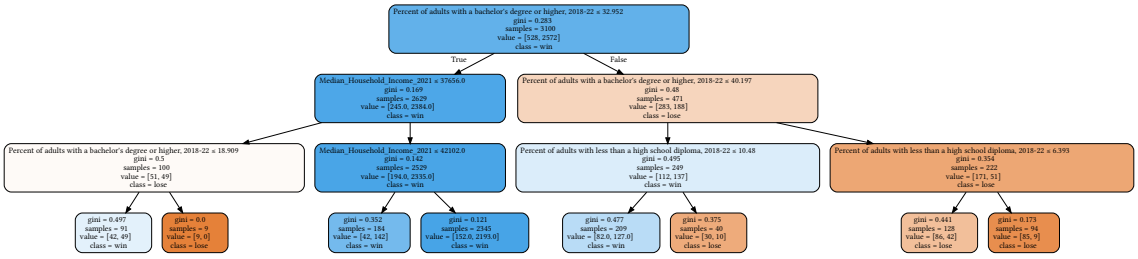


Figure 6. – Decision tree the best fitting model

3 Conclusion

Nous venons de voir rapidement la démarche générale quand on adopte le *machine learning*. Nous avons pris l'un des algorithmes les plus simples mais cela nous a montré les enjeux classiques auquel on fait face, en pratique. Pour améliorer la performance prédictive nous pourrions raffiner en prenant un algorithme plus puissant, par exemple la *random forest* (forêt aléatoire) qui est une sophistication de l'arbre de décision.

Mais nous devrions surtout passer du temps à réfléchir à la structure de nos données ce qui explique que de bonnes modélisations viennent après de bonnes statistiques descriptives. Sans ces dernières nous naviguons à vue.

Informations additionnelles

ℹ Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
```

```
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.